

Hazard Reporting 开发协作契约

> 状态：`Ready for Development`（2026-09-15；设计 AI〔项目负责人授权〕，ADR 0013）
> Owner：`A`；系统基线：`Baselined — 5d11769`；依赖波次：Wave 5
> 唯一公开入口：`package:locatemy/features/hazard\_reporting/hazard\_reporting`
> 完成定义：消费者仅凭本契约即可创建、读取、呈现及管理公共隐患、提交声明式图层，并取得可保存的附近隐患数；不会读取 `Feature` 内部状态、绕过 RLS，或把隐患混为官方治安结果。

本文件是 Hazard Reporting 唯一的跨 Owner 开发协作契约，也是同名人类 PDF 的 Markdown 源。它固定了 Dart 声明、输入约束、typed outcomes、权限/副作用、顺序、数据访问及联合验收；`lib/features/hazard\_reporting/` 内的 Widget、状态管理、Supabase/空间 Adapter、分页、缓存、并发、取消、重试、文件拆分与测试组织。公式、字段、RLS、RPC SQL 和 migration 仍分别以产品知识库与 Schema Catalog 为唯一权威。

0. 固定阅读顺序与四项 Readiness

1. [领域词汇](`../..../CONTEXT.md#项目自有表`)、[隐患产品事实](`../..../knowledge\_base/locatemy\_product/features/hazard\_reporting.md`)；
2. [Feature map (Hazard)](`../system/feature-map.md#fm-hazard`)、[Interface 注册表](`../system/interfaces.md`)、[FLOW-05](`../system/flows.md#flow-05房产实地照片风险快照与回收站`)；
3. [Capability Traceability](`../system/capability-traceability.md`)、[数据所有权](`../system/data-ownership.md`)、[Schema Catalog](`../data/schema-catalog.md`)、[风险登记](`../system/risks-and-decisions.md#风险与关闭条件`)；
4. 本契约；生成/使用人类 PDF 时最后读 [ADR 0014](`../..../adr/0014-version-locked-product-development-documentation-packages.md`) 与 [handoff](`../handoff/README.md`)。

Readiness	可核查证据	结论	
---	---	---	
责任与范围	`HAZ-01` - `HAZ-04` 唯一归属 Hazard；地图、Shell、账户 scope、官方安全与 Product snapshot 属于其他 Owner	已就绪	
契约与消费者	`HAZARD-001`、`HAZARD-002` 的唯一入口、声明、次序和 fake 在第 3 节；上游仅 `SHELL-001`、`LOCATION-002`、`PRIVACY-001`	已就绪	
数据与安全	仅使用 `crowdsourced_hazards`、`crowdsourced_hazard_votes`、`hazard_vote_counts`；Schema Catalog 定义 status-only、RLS 与安全 RPC	已就绪	
验收与风险	第 5 节覆盖 canonical `AT-HAZARD-*`、`AT-PROP-03`；`RISK-HAZARD-02` 留作实现/集成运行时证据	已就绪	

1. 成果、责任与冻结边界

- 已开启账户范围的用户可在线创建五类公共报告、按 viewport 分页读取图层与详情、对每份报告投赞成/反对/撤回，并只管理自己的 `pending/resolved` 状态或删除自己的报告。
- 发布后报告的 type、trim 后 title、description、location、report time 与 author 均不可编辑。`pending/resolved` 是作者处理标记，不是平台验证、审核、隐藏或删除决定；没有维护者例外。
- 房产只经 `HAZARD-002` 请求同一合法房产坐标周围的完整 pending 数，Hazard 不写房产风险快照。隐患绝不进 `SAFETY-001`、地点摘要或适配度。

Owner / 受控边界	负责	不负责	协作
`lib/features/hazard\_reporting/` (A)			

报告、详情、图层读取 / 分页、作者状态 / 删除、本人投票、`HAZARD-001/002` | 地图手势 / 底图、导航、账户
scope、官方犯罪、房产写入 | 消费 Shell、Map、Privacy; 提供 Hazard Interface |
| Application Shell | opened 门控、新建 / 详情 / 本人列表 / 返回导航和组合 | 报告字段、RLS、投票、计数
| `SHELL-001` |
| Map / Location | 长按产生合法创建意图、寄宿声明式图层并回传点击 | 报告读取 / 写入、内容 / 权限、详情语义 |
`LOCATION-002` |
| Account Privacy | 同账户 opened / closing 和旧账户私有状态清理证明 | 远端报告 / 投票删除或公共编辑
`PRIVACY-001` |
| Property Inspection | 何时将完整附近数写入原子风险快照 | 报告、计数口径、公共图层 | 消费
`HAZARD-002` |

2. 需要调用的 Interface

`SHELL-001` — Hazard 导航与组合

****提供者:**** Application Shell; ****消费者:**** Hazard Reporting; ****唯一公开 import:****
`package:locatemy/app/application\_shell.dart`。

```
'''dart
abstract interface class ApplicationShell {
  Future<ShellIntentOutcome> submit(ShellIntent intent);
  Future<ShellContributionOutcome> publish(ShellContribution contribution);
}
abstract interface class ShellIntent {}
abstract interface class ShellContribution {}
sealed class ShellIntentOutcome {}
final class ShellIntentAccepted extends ShellIntentOutcome {}
final class ShellAuthenticationRequired extends ShellIntentOutcome {}
final class ShellIntentRejected extends ShellIntentOutcome { final ShellRejectionReason reason; }
sealed class ShellContributionOutcome {}
final class ShellContributionAccepted extends ShellContributionOutcome {}
final class ShellContributionAuthenticationRequired extends ShellContributionOutcome {}
final class ShellContributionRejected extends ShellContributionOutcome { final ShellRejectionReason reason; }
enum ShellRejectionReason { missingInput, staleInput, inapplicableDestinationUnavailable }
'''
```

Hazard 从唯一入口导出 `OpenHazardComposerIntent`、`OpenHazardDetailIntent`、`MyHazardsIntent`、`ReturnToHazardMapIntent` (均为 `ShellIntent`) 和 `HazardShellContribution` (为 `ShellContribution`)。详情 / 返回 intent 只带稳定 `HazardReportId` 或原返回语境; 不得携带可变地图对象、另一账户作者 / 投票状态或未验证计数。

| 输入约束 | 输出 / typed failures | 状态与副作用 | 顺序、权限与最小示例 |

```
| --- | --- | --- | --- |
| intent / contribution 是当前 opened scope 的未过期事实；贡献保留来源、时间、完整性。 |
`accepted`、`authenticationRequired`、`rejected(reason)`；后两者是 Shell 经
Hazard 资料失败。 | Shell 只改门控、栈和组合；Hazard 不改报告、投票、地点或房产。 | 先收到 Map 合法
intent / stable id, 再 submit；仅 accepted 进入目标。`await
shell.submit(OpenHazardDetailIntent(id, context));` |
```

****Fake 场景：**** fake Shell 回 accepted、authentication-required、stale-in-rejected；rejected 时保留表单 / 已读页及原因，不创建 / 更新报告，不把导航拒绝显示为网络错误。

`LOCATION-002` — 长按、图层宿主与点击意图

****提供者：**** Map / Location；****消费者：**** Hazard Reporting；****唯一公开 import：****
`package:locatemy/features/map\_location/map\_location.dart`。

Hazard 只使用 Map 已定义的 `MapLayerHost.contribute`、`MapLayerContribution`、
`PlayerItem`、`CreateHazardIntent`、`ProviderDefinedIntent` 与
`MapLayer\*Outcome`。长按先由 Map 空间校验，成功才返回带 `ValidLocationReference` 的
`CreateHazardIntent`；Hazard 不把裸坐标变为可提交写入。items 必有稳定 report
id、合法坐标、viewport version 和 provider-defined detail intent。

```
| 输入约束 | 输出 / typed failures | 状态与副作用 | 顺序、权限与最小示例 |
| --- | --- | --- | --- |
| 创建只接受 Map immutable location；图层 items 为当前 viewport 成功页，id 不复用。 |
accepted、hidden、rejected(`invalidContribution/invalidCoordinate/out
alwaysia/scopeUnavailable/staleViewport/unauthenticated`)；长按结果同 Map
Map 只更新覆盖物 / 回传意图；点击、长按、贡献均不改 single/A/B/property，Hazard 不写 Map。 | 先该
contribute；旧 viewport 不发布 / 覆盖新图层。`await host.contribute(hazardLayer)`
```

****Fake 场景：**** fake Map 给合法 / 范围外长按、visible/hidden/rejected、两个 viewport
version；Hazard 仅对合法 intent 开表单，旧页不能覆盖新图层，partial 页不称范围为空。

`PRIVACY-001` — 账户范围

****提供者：**** Account Privacy；****消费者：**** Hazard Reporting；****唯一公开 import：****
`package:locatemy/features/account\_privacy/account\_privacy.dart`。

Hazard 不发起 scope open/close；只服从同账户 `opened` 和 close
请求，清理“本人报告 / 投票私有视图状态与未完成请求”。closing
开始即阻断旧账户作者 / 投票读取、提交、重放和晚到响应；完全去身份公共读缓存可留，远端报告 / 投票不因退出删除。

****Fake 场景：**** A 创建 / 投票中 close, fake Privacy 令 B opened；A
输入、本人列表、投票选择和晚到成功均不可呈现 / 提交，B 只见 B 私有状态，公共去身份页可重新读取。

3. 必须提供的 Interface

`HAZARD-001` — 公共报告、作者管理、投票与图层

****提供者:**** Hazard Reporting (A) ; ****消费者:**** Application Shell、Map / Location ; ****唯一公开 import:**** `package:locatemy/features/hazard\_reporting/hazard\_reporting.dart`。

消费者只能 `import` 此入口。A 应先合入以下声明与最小 `fake` ; 它们是协作形状, 不是可提交实现体、SQL 或 SDK 映射。

```
'''dart
abstract interface class HazardReporting {
  Future<HazardCreateOutcome> create(HazardCreateRequest request);
  Future<HazardPageOutcome> loadPublic(HazardPageRequest request);
  Future<HazardDetailOutcome> loadDetail(HazardReportId id);
  Future<MyHazardsOutcome> loadMine(HazardPageRequest request);
  Future<HazardStatusOutcome> changeMyStatus(HazardStatusRequest request);
  Future<HazardDeleteOutcome> deleteMine(HazardReportId id);
  Future<HazardVoteOutcome> vote(HazardVoteRequest request);
}

enum HazardType { flood, crime, traffic, infrastructure, other }
enum HazardAuthorStatus { pending, resolved }
enum HazardVote { up, down, none }

final class HazardReportId { final String value; const HazardReportId(
final class HazardCreateRequest { final ValidLocationReference location;
type; final String title; final String? description; const HazardCr
this.location, required this.type, required this.title, this.descri
final class HazardPageRequest { final String viewportVersion; final Ha
final String? cursor; const HazardPageRequest({required this.viewportV
this.viewport, this.cursor}); }

final class HazardViewport { final GeographicPoint southWest; final Ge
northEast; const HazardViewport(this.southWest, this.northEast); }

final class HazardStatusRequest { final HazardReportId id; final Hazar
const HazardStatusRequest(this.id, this.status); }

final class HazardVoteRequest { final HazardReportId id; final HazardV
HazardVoteRequest(this.id, this.vote); }

final class HazardReport { final HazardReportId id; final HazardType t
title; final String? description; final GeographicPoint location; f
status; final DateTime reportedAt; final HazardAuthorView author; f
vote; const HazardReport({required this.id, required this.type, req
this.description, required this.location, required this.status, req
required this.author, required this.vote}); }

enum HazardAuthorView { mine, other }

final class HazardVoteState { final HazardVote mine; final int upvotes
const HazardVoteState(this.mine, this.upvotes, this.downvotes); }

sealed class HazardCreateOutcome { const HazardCreateOutcome(); }
final class HazardCreated extends HazardCreateOutcome { final HazardRe
HazardCreated(this.report); }

final class HazardCreateRejected extends HazardCreateOutcome { final H
failure; const HazardCreateRejected(this.failure); }

sealed class HazardPageOutcome { const HazardPageOutcome(); }
final class HazardPageAvailable extends HazardPageOutcome { final Haza
```

```

HazardPageAvailable(this.page); }

final class HazardPagePartial extends HazardPageOutcome { final HazardReadFailure failure; const HazardPagePartial(this.page, this.failure); }

final class HazardPageUnavailable extends HazardPageOutcome { final HazardReadFailure failure; const HazardPageUnavailable(this.failure); }

final class HazardPage { final List<HazardReport> reports; final String viewportVersion; const HazardPage(this.reports, this.nextCursor, this.viewportVersion); }

sealed class HazardDetailOutcome { const HazardDetailOutcome(); }

final class HazardDetailAvailable extends HazardDetailOutcome { final HazardReport report; const HazardDetailAvailable(this.report); }

final class HazardDetailUnavailable extends HazardDetailOutcome { final HazardReadFailure failure; const HazardDetailUnavailable(this.failure); }

sealed class MyHazardsOutcome { const MyHazardsOutcome(); }

final class MyHazardsAvailable extends MyHazardsOutcome { final List<HazardReport> reports; const MyHazardsAvailable(this.reports); }

final class MyHazardsUnavailable extends MyHazardsOutcome { final HazardReadFailure failure; const MyHazardsUnavailable(this.failure); }

sealed class HazardStatusOutcome { const HazardStatusOutcome(); }

final class HazardStatusChanged extends HazardStatusOutcome { final HazardReport report; const HazardStatusChanged(this.report); }

final class HazardStatusRejected extends HazardStatusOutcome { final HazardReadFailure failure; const HazardStatusRejected(this.failure); }

sealed class HazardDeleteOutcome { const HazardDeleteOutcome(); }

final class HazardDeleted extends HazardDeleteOutcome { final HazardReport report; const HazardDeleted(this.id); }

final class HazardDeleteRejected extends HazardDeleteOutcome { final HazardReadFailure failure; const HazardDeleteRejected(this.failure); }

sealed class HazardVoteOutcome { const HazardVoteOutcome(); }

final class HazardVoteChanged extends HazardVoteOutcome { final HazardReport report; final HazardVoteState state; const HazardVoteChanged(this.id, this.state); }

final class HazardVoteRejected extends HazardVoteOutcome { final HazardReadFailure failure; const HazardVoteRejected(this.failure); }

enum HazardReadFailure { authenticationRequired, invalidViewport, notFound, retryableUnavailable, incompletePage, scopeUnavailable }

enum HazardWriteFailure { invalidType, emptyTitle, titleTooLong, descriptionTooLong, invalidLocation, authenticationRequired, permissionDenied, conflict, retryableUnavailable, scopeUnavailable, immutableContent }

```

调用	输入约束	typed output / failure	状态、副作用、权限与顺序
`create`	opened; type 必选; title trim 后 1-120; description 可空且 ≤2000; location 是 Map immutable reference; 在线。	`HazardCreated` 或字段 / 权限 / 服务分类的 `HazardCreateRejected`。	author-only insert; 成功后权威 `reportedAt/author/status` 不由客户端改写。失败留输入; 无离线队列 / 伪 created。
`loadPublic` / `loadDetail`	authenticated; 有效 viewport/cursor 或 id; 旧 viewport 不采用。	available、partial (成功页加 failure)、unavailable; 空	

reports 仅成功页为空。 | authenticated 公读；先完成当前页再贡献图层，不能把失败 / partial 置 0 或范
|
| `loadMine` | opened；请求只代表当前账户。 | available 或
unavailable；不以空列表表示权限 / 网络失败。 | 仅当前 author；close 后丢弃响应。 |
| `changeMyStatus` | opened；稳定 id；仅 `pending/resolved`。 | changed (状态
report) 或 rejected。 | 仅 author update
status；type/title/description/location/reportedAt/author 无编辑路径，写入为
immutableContent/permission failure。成功后详情、mine、图层一致刷新。 |
| `deleteMine` | opened；稳定 id；用户已确认。 | deleted 或 rejected。 | 仅 author
delete；取消无副作用；级联本人 votes；并发 delete 为 notFound/conflict，不假称成功。 |
| `vote` | opened；稳定 id；`up/down/none`；一账户一报告。 | changed (本人选择与 aggregation
counts) 或 rejected。 | 只写本人 vote；none 撤回。counts 只经 `hazard\_vote\_count`
RPC 读，客户端不写 count、不读他人 vote / 身份。 |

****最小调用：**** `await hazards.create(HazardCreateRequest(location:
intent.location, type: HazardType.flood, title: title, description:
description));`。只有 `HazardCreated` 才 publish 图层 / 详情贡献；其他 outcome
原样进入字段或恢复路径。

****Fake 场景：**** Shell fake 消费 created、empty successful
page、partial、authentication-required、permission、conflict、retryable；
successful empty 显示“暂无”。Map fake 验证 item 只有 id/location/provider-de
intent, 旧 viewport 不发布。双账户 fake 改票 / 撤回并验证同一 counts、无匿名 / 他人 vote 枚举。aut
fake 验证仅本人 status/delete, A close 后成功不可落入 B。

`HAZARD-002` — 房产风险附近 pending 数

****提供者：**** Hazard Reporting (A)；****消费者：**** Property Inspection；****唯一公开**
import:** `package:locatemy/features/hazard_reporting/hazard_report
art`。

```
'''dart
abstract interface class HazardRiskCounter { Future<HazardNearbyCountOut
countPending(HazardNearbyCountRequest request); }
final class HazardNearbyCountRequest { final ValidLocationReference pr
HazardNearbyCountRequest(this.propertyLocation); }
sealed class HazardNearbyCountOutcome { const HazardNearbyCountOutcome
final class HazardNearbyCountAvailable extends HazardNearbyCountOutcom
final int radiusMeters; final DateTime countedAt; const
HazardNearbyCountAvailable(this.count, this.radiusMeters, this.coun
final class HazardNearbyCountUnavailable extends HazardNearbyCountOutc
HazardNearbyCountFailure failure; const HazardNearbyCountUnavailabl
enum HazardNearbyCountFailure { invalidLocation, authenticationRequire
retryableUnavailable, scopeUnavailable }
'''
```

输入约束	output / failure	状态与副作用	顺序、权限与最小示例

| propertyLocation 是 `LOCATION-001` 合法 immutable property reference available 带 count/radiusMeters/countedAt;unavailable 区分 invalid、partial、scope、retryable。 | 只读公开 reports;不写报告、票、图层、官方安全或 Pro snapshot。 | Haversine `d <= 2,000m` 含边界;仅 pending。Property 仅在同坐标完整 `SAFETY-001` 也 available 时整体写 snapshot。`await counter.countPending(HazardNearbyCountRequest(location));` |

Fake 场景: Property fake 提供内/外/恰 2,000m、pending/resolved;仅内/边界 pending 计入。partial/failure 不返回 0, Property 保留旧原子快照并显示不可用。

4. 数据与固定流程

目的	权威对象 / 精确访问	固定语义		
报告		`crowdsourced\_hazards`;authenticated 读, author-only insert/delete/status update	五类、WGS84、pending/resolved、report time;发布后内容/位置/时间/author 不可变;无审核/verified/rejected/维护者例外。	
投票		`crowdsourced\_hazard\_votes`;本人行读写、撤回删除		`(hazard\_id,user\_id)` 唯一;`-1/+1`;一账户一票。
计数		`hazard\_vote\_counts`;authenticated 执行的受控 `SECURITY DEFINER` 仅 hazard_id/upvotes/downvotes;安全/空 search_path、调用者验证、撤销 anon/default execute;不返回投票者身份。		
私有本机状态		`PRIVACY-001` 与 [data ownership](../system/data-ownership.md#privacy-barrier-参与者清单)		
mine/vote view 与未完成请求按账户隔离并 close 清理;完全去身份公共缓存可留;业务写全在线。 |

固定顺序:① Shell 确认 opened;② 入口或 Map 合法长按经 `SHELL-001.submit` 开 compose create 写远端,成功才 publish 图层/详情;④ public page 成功后 `LOCATION-002.contribute`,点击稳定 id 经 Shell 开详情;⑤ status/delete/vote outcome 再刷新;⑥ close 即阻断私有操作与晚到响应;⑦ Property 仅消费 HAZARD-002 complete available,和同坐标 SAFETY-001 一起自行保存。对应 `FLOW-05/06`。

5. 联合验收、Ready Gate 与变更

Capability / canonical AT	情景与操作	可观察完成条件
`HAZ-01` / `AT-HAZARD-01`、`AT-HAZARD-05`	入口和合法长按创建;未选 type、trim 空/超长、非法点、离线	五类不预选;字段错误明确;online 成功才 created;失败留输入/重试,无离线伪队列。
`HAZ-01`、`HAZ-04` / `AT-HAZARD-01`、`AT-HAZARD-03`	A status/delete;B/维护者尝试更新/改已发布字段	仅 A pending/resolved 与确认删除成功;内容/位置/时间/author 不可变;状态非审核,取消无副作用。
`HAZ-02` / `AT-HAZARD-01`、`AT-HAZARD-02`	viewport、多页、点击、后页失败	图层/详情远端;empty/partial/failure 分开,成功页保留;点击/长按不改分析地点。
`HAZ-03` / `AT-HAZARD-03`、`AT-HAZARD-04`	两账户投票、改票/撤回;匿名/无效 call	一账户一票、counts 一致;枚举他票和无身份被拒;冲突/删除不虚构 count。
`HAZ-04` / `AT-HAZARD-01`、`AT-HAZARD-03`	mine、定位、A close→B open	仅当前 author 管理;返回语境保留;A 私有状态不呈现/提交/重放到 B。
`HAZARD-002` / `AT-PROP-03`	内/外/恰	

2km、pending/resolved、partial/failure | 只计含边界
pending；带半径/时间/available；partial/failure 非 0，不写部分 snapshot。 |

- [x] `HAZ-01`-`04`、`HAZARD-001/002`、D18-D20、Schema、`FLOW-05/06`、ca al `AT-\*` 可双向追踪。
- [x] 唯一 import、声明、输入、typed outputs/failures、不可变、作者 status/delete、投票/RLS、账户隔离、顺序、示例/fake 均冻结；无函数体、SQL 或 SDK 实现。
- [x] `RISK-HAZARD-01` 两账户/匿名/search-path 与 `RISK-HAZARD-02` status migration/跨视图证据为实现/集成 Gate，不阻塞 Ready。
- [x] 独立 Standards/Spec 双轴复审通过；设计 AI 依 ADR 0013 批准 Ready。

公共 Interface 变更由提供方说明原因和受影响消费者，所有受影响消费者确认；同一 PR 更新声明、本契约、受影响 fake/Adapter 测试及 PDF。Git/PR 保存历史；不使用文档版本、checksum、Manifest、Generati Gate 或 Development Release。

日期	状态	变更原因	受影响对象	批准者	
---	---	---	---	---	
2026-09-14	`Ready for Development`	Issue #14：冻结五类公共报告、发布后不可变、status/delete、单票和 2,000m pending-only count	`HAZ-01`-`04`、`HAZARD-001/002`、Hazard 数据对象、Property	设计 AI（项目负责人授权）	
2026-09-15	`Ready for Development`	Issue #23：单一契约优先 Markdown/PDF；采用最新 Shell submit/publish，补齐声明、调用卡、fake、隐私/DB/RPC 边界；不改变产品/数据模型	`HAZ-01`-`04`、`HAZARD-001/002`、`SHELL-001`、`LOCATIO`、`PRIVACY-001`	设计 AI（项目负责人授权）	